

Skiplist

跳表

跳表 (Skip List) 是由 William Pugh 发明的一种查找数据结构，支持对数据的快速查找，插入和删除。

跳表的期望空间复杂度为 $O(n)$ ，跳表的查询，插入和删除操作的期望时间复杂度都为 $O(\log n)$ 。

基本思想

顾名思义，跳表是一种类似于链表的数据结构。更加准确地说，跳表是对有序链表的改进。

为方便讨论，后续所有有序链表默认为 **升序** 排序。

一个有序链表的查找操作，就是从头部开始逐个比较，直到当前节点的值大于或者等于目标节点的值。很明显，这个操作的复杂度是 $O(n)$ 。

跳表在有序链表的基础上，引入了 **分层** 的概念。首先，跳表的每一层都是一个有序链表，特别地，最底层是初始的有序链表。每个位于第 i 层的节点有 $\frac{1}{2}$ 的概率出现在第 $i+1$ 层， $\frac{1}{2}$ 为常数。

记在 n 个节点的跳表中，期望包含 $\log n$ 个元素的层为第 $\log n$ 层，易得。

在跳表中查找，就是从第 $\log n$ 层开始，水平地逐个比较直至当前节点的下一个节点大于等于目标节点，然后移动至下一层。重复这个过程直至到达第一层且无法继续进行操作。此时，若下一个节点是目标节点，则成功查找；反之，则元素不存在。这样一来，查找的过程中会跳过一些没有必要的比较，所以相比于有序链表的查询，跳表的查询更快。可以证明，跳表查询的平均复杂度为 $O(\log n)$ 。

复杂度证明

空间复杂度

对于一个节点而言，节点的最高层数为 $\log n$ 的概率为 $\frac{1}{2^{\log n}}$ 。所以，跳表的期望层数为 $\log n$ ，且因为 $\frac{1}{2}$ 为常数，所以跳表的 **期望空间复杂度** 为 $O(n)$ 。

在最坏的情况下，每一层有序链表等于初始有序链表，即跳表的 **最差空间复杂度** 为 $O(n)$ 。

时间复杂度

从后向前分析查找路径，这个过程可以分为从最底层爬到第 $\log n$ 层和后续操作两个部分。在分析时，假设一个节点的具体信息在它被访问之前是未知的。

假设当前我们处于一个第 i 层的节点，我们并不知道 i 层的最大层数和左侧节点的最大层数，只知道 i 层的最大层数至少为 $\log n$ 。如果 i 层的最大层数大于 $\log n$ ，那么下一步应该是向上走，这种情况的概率为 $\frac{1}{2}$ ；如果 i 层的最大层数等于 $\log n$ ，那么下一步应该是向左走，这种情况概率为 $\frac{1}{2}$ 。

令 T_i 为在一个无限长度的跳表中向上爬 i 层的期望代价，那么有：

解得 $T_i = 2i$ 。

由此可以得出：在长度为 n 的跳表中，从最底层爬到第 $\log n$ 层的期望步数存在上界 $O(\log n)$ 。

现在只需要分析爬到第 $\log n$ 层后还要再走多少步。易得，到了第 $\log n$ 层后，向左走的步数不会超过第 $\log n$ 层及更高层的节点数总

和，而这个总和的期望为 $\frac{1}{2}$ 。所以到了第 l 层后向左走的期望步数存在上界 $\frac{1}{2}$ 。同理，到了第 l 层后向上走的期望步数存在上界 $\frac{1}{2}$ 。

所以，跳表查询的期望查找步数为 $\frac{1}{2}$ ，又因为 $\frac{1}{2}$ ，所以跳表查询的 **期望时间复杂度** 为 $O(\log n)$ 。

在最坏的情况下，每一层有序链表等于初始有序链表，查找过程相当于对最高层的有序链表进行查询，即跳表查询操作的 **最差时间复杂度** 为 $O(n)$ 。

插入操作和删除操作就是进行一遍查询的过程，途中记录需要修改的节点，最后完成修改。易得每一层至多只需要修改一个节点，又因为跳表期望层数为 $\frac{1}{2}$ ，所以插入和修改的 **期望时间复杂度** 也为 $O(\log n)$ 。

具体实现

获取节点的最大层数

模拟以 $\frac{1}{2}$ 的概率往上加一层，最后和上限值取最小。

```
1 int randomLevel() {
2     int lv = 1;
3     // MAXL = 32, S = 0xFFFF, PS = S * P, P = 1 / 4
4     while ((rand() & S) < PS) ++lv;
5     return min(MAXL, lv);
6 }
```

查询

查询跳表中是否存在键值为 `key` 的节点。具体实现时，可以设置两个哨兵节点以减少边界条件的讨论。

```
1 V& find(const K& key) {
2     SkipListNode<K, V>* p = head;
3
4     // 找到该层最后一个键值小于 key 的节点，然后走向下一层
5     for (int i = level; i >= 0; --i) {
6         while (p->forward[i]->key < key) {
7             p = p->forward[i];
8         }
9     }
10    // 现在是小于，所以还需要再往后走一步
11    p = p->forward[0];
12
13    // 成功找到节点
14    if (p->key == key) return p->value;
15
16    // 节点不存在，返回 INVALID
17    return tail->value;
18 }
```

插入

插入节点 (`key`, `value`)。插入节点的过程就是先执行一遍查询的过程，中途记录新节点是要插入哪一些节点的后面，最后再执行插入。每一层最后一个键值小于 `key` 的节点，就是需要进行修改的节点。

```

1 void insert(const K &key, const V &value) {
2     // 用于记录需要修改的节点
3     SkipListNode<K, V> *update[MAXL + 1];
4
5     SkipListNode<K, V> *p = head;
6     for (int i = level; i >= 0; --i) {
7         while (p->forward[i]->key < key) {
8             p = p->forward[i];
9         }
10        // 第 i 层需要修改的节点为 p
11        update[i] = p;
12    }
13    p = p->forward[0];
14
15    // 若已存在则修改
16    if (p->key == key) {
17        p->value = value;
18        return;
19    }
20
21    // 获取新节点的最大层数
22    int lv = randomLevel();
23    if (lv > level) {
24        lv = ++level;
25        update[lv] = head;
26    }
27
28    // 新建节点
29    SkipListNode<K, V> *newNode = new SkipListNode<K, V>(key, value, lv);
30    // 在第 0~lv 层插入新节点
31    for (int i = lv; i >= 0; --i) {
32        p = update[i];
33        newNode->forward[i] = p->forward[i];
34        p->forward[i] = newNode;
35    }
36
37    ++length;
38 }

```

删除

删除键值为 `key` 的节点。删除节点的过程就是先执行一遍查询的过程，中途记录要删的节点是在哪一些节点的后面，最后再执行删除。每一层最后一个键值小于 `key` 的节点，就是需要进行修改的节点。

```

1 bool erase(const K &key) {
2     // 用于记录需要修改的节点
3     SkipListNode<K, V> *update[MAXL + 1];
4
5     SkipListNode<K, V> *p = head;
6     for (int i = level; i >= 0; --i) {
7         while (p->forward[i]->key < key) {
8             p = p->forward[i];
9         }
10        // 第 i 层需要修改的节点为 p
11        update[i] = p;
12    }
13    p = p->forward[0];
14
15    // 节点不存在
16    if (p->key != key) return false;
17
18    // 从最底层开始删除
19    for (int i = 0; i <= level; ++i) {
20        // 如果这层没有 p 删除就完成了
21        if (update[i]->forward[i] != p) {
22            break;
23        }
24        // 断开 p 的连接
25        update[i]->forward[i] = p->forward[i];
26    }
27
28    // 回收空间
29    delete p;
30
31    // 删除节点可能导致最大层数减少
32    while (level > 0 && head->forward[level] == tail) --level;
33
34    // 跳表长度
35    --length;
36    return true;
37 }

```

完整代码

下列代码是用跳表实现的 map。未经正经测试，仅供参考。

► 参考代码

跳表的随机访问优化

访问跳表中第 i 个节点，相当于访问初始有序链表中的第 i 个节点，很明显这个操作的时间复杂度是 $O(n)$ 的，并不足够优秀。

跳表的随机访问优化就是对每一个前向指针，再多维护这个前向指针的长度。假设 p 和 q 都是跳表中的节点，其中 p 为跳表的第 i 个节点， q 为跳表的第 j 个节点，且在跳表的某一层中 p 的前向指针指向 q ，那么这个前向指针的长度为 $j - i$ 。

现在访问跳表中的第 i 个节点，就可以从顶层开始，水平地遍历该层的链表，直到当前节点的位置加上当前节点在该层的前向指针长度大于等于 i ，然后移动至下一层。重复这个过程直至到达第一层且无法继续行操作。此时，当前节点就是跳表中第 i 个节点。

这样，就可以快速地访问到跳表的第 i 个元素。可以证明，这个操作的时间复杂度为 $O(\log n)$ 。

参考资料

1. [Skip Lists: A Probabilistic Alternative to Balanced Trees](#)
 2. [Skip List](#)
 3. [A Skip List Cookbook](#)
-

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [Backlight](#), [CookiePieWw](#), [Enter-tainer](#), [ksyx](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用